Confluent Cloud Python Protobuf CSFLE (Client-Side Field-Level Encryption) Example

A hands-on Python demonstration of the **Confluent Cloud Protobuf Schema Serializer & Deserializer**, covering every major concept from the official Confluent Protobuf SerDes documentation.

The project talks to a Confluent Cloud Schema Registry over the SR REST API and, when run in full mode, produces and consumes messages on a Kafka cluster via `confluent-kafka`. No `protoc` compiler or generated stubs are required — schemas are defined as Python dataclasses, compiled at runtime into `google.protobuf FileDescriptorProto` objects, and serialized to **Protobuf binary** using `google.protobuf.message_factory`.

Table of Contents

- 1.0 Anatomy & Setup
  - 1.1 Project layout
  - 1.2 Architecture Overview
  - 1.3 Requirements
  - 1.4 Setup
  - 1.5 Dependencies
  - 1.6 Configuration
  - 1.7 Confluent Cloud prerequisites
  - 1.8 Core classes
    - 1.8.1 `SchemaRegistryClient` (`schema_registry_client.py`)
    - 1.8.2 `ProtoMessage` / `ProtoField` (`dynamic_protobuf_helpers.py`)
    - 1.8.3 `KafkaProtobufSerializer` (`kafka_protobuf_serdes.py`)
    - 1.8.4 `KafkaProtobufDeserializer` (`kafka_protobuf_serdes.py`)
    - 1.8.5 `FieldEncryptor` / `get_encrypted_fields()` (`field_encryption.py`)
    - 1.8.6 `kafka_helpers.py`
    - 1.8.7 `utilities.py`
    - 1.8.8 `demos.py`
  - 1.9 Logging
  - 1.10 Wire format
    - 1.10.1 Why a message index?
    - 1.10.2 What the code does
- 2.0 Running the demos
  - 2.1 Demos
    - 2.1.1 Demo 1 — Basic Serializer & Deserializer (`--demo basic`)
    - 2.1.2 Demo 2 — Reference-Deletion Protection (`--demo delete-protection`)
    - 2.1.3 Demo 3 — Schema Evolution (`--demo evolution`)
    - 2.1.4 Demo 4 — Multiple Event Types / `oneOf` (`--demo oneof`)
    - 2.1.5 Demo 5 — Null-Value Handling (`--demo null-handling`)
    - 2.1.6 Demo 6 — Compatibility Rules (`--demo compatibility`)
    - 2.1.7 Demo 7 — Schema Types (`--demo types`)

- 2.1.8 Demo 8 — Subject Name Strategies (`--demo strategies`)
- 2.1.9 Demo 9 — Client-Side Field Level Encryption (`--demo csfle`)

- 3.0 Cleanup
- 4.0 Notes
- 5.0 Resources

1.0 Anatomy & Setup

1.1 Project layout

```
cc-python-protobuf-csfle-example/
├── src
│   ├── constants.py                # DEFAULT_TOOL_LOG_FILE,
DEFAULT_TOOL_LOG_FORMAT
│   ├── utilities.py                # setup_logging(), get_config(),
parse_args() — logging, env config, CLI
│   ├── schema_registry_client.py   # SchemaRegistryClient — SR REST API
wrapper + wire format
│   ├── kafka_protobuf_serdes.py    # KafkaProtobufSerializer &
KafkaProtobufDeserializer
│   ├── kafka_helpers.py            # ensure_topics(), kafka_produce(),
kafka_consume_one()
│   ├── dynamic_protobuf_helpers.py # ProtoMessage & ProtoField — dynamic
proto3 schema builders
│   ├── field_encryption.py         # FieldEncryptor &
get_encrypted_fields() — AES-256-GCM CSFLE
│   ├── demos.py                    # All nine demo functions (demo_basic
… demo_csfle)
│   └── main.py                     # Thin entry point — wires config, SR
client, and demo dispatch
├── run-all-demos.sh                # Shell script — authenticates via
AWS SSO and runs all demos in full mode
├── pyproject.toml                  # Project metadata, dependencies,
logging
├── uv.lock                         # Pinned dependency lockfile — commit
this
├── .env                            # Credentials — NOT COMMITTED, loaded
automatically by python-dotenv at startup
├── .gitignore                      # Ignore .env, .venv/, logs,
__pycache__/, etc.
├── CHANGELOG.md                    # Changelog in Markdown format
├── CHANGELOG.pdf                   # Changelog in PDF format
├── KNOWNISSUES.md                  # Known issues in Markdown format
├── KNOWNISSUES.pdf                 # Known issues in PDF format
├── LICENSE.md                      # License in Markdown format
├── LICENSE.pdf                     # License in PDF format
└── README.md                       # This file — project overview, setup
instructions, and documentation of all core concepts and classes
```

1.2 Architecture Overview

```
flowchart TB

    %% ── Bootstrap ──────────────────────────────────────────────────────
    subgraph Boot["Startup"]
        direction LR
        ENV[".env file"]
        PYPROJECT["pyproject.toml\n[tool.logging]"]

CONST["constants.py\nDEFAULT_TOOL_LOG_FILE\nDEFAULT_TOOL_LOG_FORMAT"]
        UTIL["utilities.py\nsetup_logging()\ntomllib.load()"]
        LOG["logging.config\n.dictConfig()"]
        DOTENV["load_dotenv()"]

        CONST --> UTIL
        PYPROJECT --> UTIL
        UTIL --> LOG
        ENV --> DOTENV
    end

    %% ── CLI ────────────────────────────────────────────────────────────
    subgraph CLI["CLI  (main.py · utilities.py)"]
        direction LR
        ARGS["parse_args()\n--mode  --demo  --run-id"]
        CFG["get_config()\nreads os.environ"]
        ARGS --> CFG
    end

    Boot --> CLI

    %% ── Schema helpers ─────────────────────────────────────────────────
    subgraph SCHEMA["Proto Schema Helpers  (dynamic_protobuf_helpers.py)"]
        direction TB
        PF["ProtoField\nname · type · number\noptional · repeated"]
        PM["ProtoMessage\nname · package · imports\nfields · oneofs"]
        TSS["to_schema_string()\n→ proto3 text"]
        SER["serialize(data)\nParseDict → SerializeToString\n→ Protobuf
binary"]
        DES["deserialize(raw)\nFromString → MessageToDict\n→ dict"]
        PM --> TSS
        PM --> SER
        PM --> DES
        PF --> PM
    end

    %% ── SR client
──────────────────────────────────────────────────────────
    subgraph SRC["SchemaRegistryClient  (schema_registry_client.py)"]
        direction TB
        SESS["requests.Session\nBasic auth (SR key/secret)\nContent-Type:
vnd.schemaregistry.v1+json"]
```

```
        CACHE["_cache: dict[int, dict]\nschema_id → schema_obj"]

        subgraph SR_READ["Read"]
            GET_TYPES["GET /schemas/types"]
            GET_SUBJ["GET /subjects"]
            GET_VERS["GET /subjects/{s}/versions"]
            GET_VER["GET /subjects/{s}/versions/{v}"]
            GET_ID["GET /schemas/ids/{id}  ← cached"]
            GET_ID_VERS["GET /schemas/ids/{id}/versions"]
            GET_REFBY["GET /subjects/{s}/versions/{v}/referencedby"]
        end

        subgraph SR_WRITE["Write / Delete"]
            POST_REG["POST /subjects/{s}/versions\nregister schema →
schema_id"]
            DEL_VER["DELETE /subjects/{s}/versions/{v}"]
            DEL_SUBJ["DELETE /subjects/{s}  soft delete"]
            DEL_PERM["DELETE /subjects/{s}?permanent=true"]
        end

        subgraph SR_COMPAT["Compatibility"]
            POST_COMPAT["POST /compatibility/subjects/{s}/versions/{v}\n→
is_compatible"]
            GET_CFG["GET /config[/{s}]\n→ compatibilityLevel"]
            PUT_CFG["PUT /config[/{s}]\nset level e.g.
BACKWARD_TRANSITIVE"]
        end

        subgraph WIRE["Wire Format  (local)"]
            ENC["encode(schema_id, payload)\nstruct.pack('>bI', 0x00, id)
+ payload"]
            DEC["decode_header(data)\nvalidate magic
0x00\nstruct.unpack('>I', data[1:5])"]
        end

        SESS --> SR_READ
        SESS --> SR_WRITE
        SESS --> SR_COMPAT
        GET_ID --> CACHE
    end

    %% ── SerDes
──────────────────────────────────────────────────────────────
    subgraph SERDES["SerDes  (kafka_protobuf_serdes.py)"]
        direction LR
        subgraph KSER["KafkaProtobufSerializer"]
            STRAT["SubjectNameStrategy\nTopicName · 
RecordName\nTopicRecordName"]
            REFSTRAT["ReferenceSubjectNameStrategy\nDefault · Qualified"]
            AUTO["auto_register=True\n→ sr.register()\nor
sr.get_version()"]
            KSER_OUT["sr.encode(schema_id, payload)\n→ wire bytes"]
            STRAT --> AUTO
            AUTO --> KSER_OUT
```

```
        end
        subgraph KDES["KafkaProtobufDeserializer"]
            KDES_IN["sr.decode_header(raw)\n→ schema_id + payload"]
            WARM["sr.get_schema_by_id()\nwarm cache"]
            SPEC["specific_type set?\n→ specific_type.deserialize()\nelse
_schema_id_to_message lookup"]
            KDES_IN --> WARM
            WARM --> SPEC
        end
    end

    %% ── Kafka helpers
──────────────────────────────────────────────────
    subgraph KAFKA_H["Kafka Helpers  (kafka_helpers.py · --mode full
only)"]
        direction TB

BASE["_base_kafka_config()\nbootstrap.servers\nsecurity.protocol=SASL_SSL\
nsasl.mechanisms=PLAIN"]
        ADMIN_H["ensure_topics()\nAdminClient →
list_topics()\nNewTopic(rf=3, partitions=6)\ncreate_topics()  idempotent"]
        PROD_H["kafka_produce()\nProducer → produce() + flush()"]
        CONS_H["kafka_consume_one()\nConsumer → subscribe()\npoll() loop +
commit()"]
        BASE --> ADMIN_H
        BASE --> PROD_H
        BASE --> CONS_H
    end

    %% ── CSFLE ──────────────────────────────────────────────────────────
    subgraph CSFLE_BOX["CSFLE  (field_encryption.py)"]
        direction TB
        FE["FieldEncryptor\nAES-256-GCM\nAWS KMS + DEK Registry"]
        GEF["get_encrypted_fields()\nmetadata tags → field list"]
        GEF --> FE
    end

    %% ── Demos
──────────────────────────────────────────────────────────
    subgraph DEMOS["Demo Sections  (demos.py)"]
        direction LR
        D1["Demo 1 · Basic\ntestproto-{run_id}"]
        D2["Demo 2 · Delete Protection\nreferencedby → correct order"]
        D3["Demo 3 · Schema Evolution\nBACKWARD_TRANSITIVE\ntransactions-
proto-{run_id}"]
        D4["Demo 4 · oneOf\nAllTypes wrapper\nall-events-{run_id}"]
        D5["Demo 5 · Null Handling\noptional fields\nnullables-{run_id}"]
        D6["Demo 6 · Compatibility\n7 modes reference table"]
        D7["Demo 7 · Schema Types\nspecific vs DynamicMessage"]
        D8["Demo 8 · Subject Strategies\npayments-{run_id}"]
        D9["Demo 9 · CSFLE\nfield-level encryption\ncsfle-{run_id}"]
    end

    %% ── Confluent Cloud
```

```
────────────────────────────────────────────────────────────
    subgraph CC["Confluent Cloud"]
        CC_SR["Schema Registry\nStream Governance\nEssentials / Advanced"]
        CC_KAFKA["Kafka Cluster\nSASL_SSL · PLAIN\nrf=3"]
    end

    %% ── Wiring
────────────────────────────────────────────────────────────
    CLI --> SRC
    CLI --> SCHEMA
    CLI --> SERDES
    CLI --> KAFKA_H
    CLI --> DEMOS
    CLI --> CSFLE_BOX

    SCHEMA --> SERDES
    SRC --> SERDES
    SRC --> DEMOS
    SERDES --> DEMOS
    KAFKA_H --> DEMOS

    CSFLE_BOX --> SERDES
    CSFLE_BOX --> DEMOS

    SRC  -->|"HTTPS REST"| CC_SR
    KAFKA_H -->|"SASL_SSL"| CC_KAFKA

    %% ── Styles
────────────────────────────────────────────────────────────
    classDef cloud    fill:#0a74da,color:#fff,stroke:#0a74da
    classDef core     fill:#1a6b3c,color:#fff,stroke:#1a6b3c
    classDef helper   fill:#6b4226,color:#fff,stroke:#6b4226
    classDef demo     fill:#5a3472,color:#fff,stroke:#5a3472
    classDef wire     fill:#b85c00,color:#fff,stroke:#b85c00
    classDef boot     fill:#2b4a6b,color:#fff,stroke:#2b4a6b

    class CC_SR,CC_KAFKA cloud
    class SRC,SERDES core
    class KAFKA_H,SCHEMA helper
    class DEMOS,D1,D2,D3,D4,D5,D6,D7,D8,D9 demo
    class WIRE,ENC,DEC wire
    class Boot,UTIL,CONST boot
    classDef csfle    fill:#8b1a1a,color:#fff,stroke:#8b1a1a
    class CSFLE_BOX,FE,GEF csfle
```

1.3 Requirements

- **Python ≥ 3.13**
- **uv** — package and project manager

1.3.1 Install uv

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# or via Homebrew
brew install uv
```

1.4 Setup

```
git clone <repo-url>
cd cc-python-protobuf-csfle-example

# Create .venv and install exact pinned versions from uv.lock
uv sync
```

`uv sync` reads both `pyproject.toml` and `uv.lock` and installs everything into a local `.venv`. No manual `pip install` is needed.

1.5 Dependencies

Package	Minimum	Purpose
`boto3`	1.38.0	AWS KMS client for CSFLE DEK unwrapping
`confluent-kafka`	2.13.2	Producer, Consumer, AdminClient (required for `--mode full` only)
`cryptography`	44.0.0	AES-256-GCM encryption for Client-Side Field Level Encryption (CSFLE)
`protobuf`	7.34.0	`google.protobuf` runtime (available for real binary encoding)
`requests`	2.32.5	Schema Registry REST API calls
`python-dotenv`	1.2.2	Auto-loads `.env` via `load_dotenv()` at startup
`dotenv`	0.9.9	dotenv compatibility shim

> `confluent-kafka` is imported inside a `try/except` at startup; if it is absent the app still runs normally in `--mode schema-only`.

1.6 Configuration

Create a `.env` file in the project root (never commit it):

```
# Kafka cluster — only required for --mode full
BOOTSTRAP_SERVERS=pkc-xxxxx.us-east1.gcp.confluent.cloud:9092
KAFKA_API_KEY=your_kafka_api_key
```

```
KAFKA_API_SECRET=your_kafka_api_secret

# Schema Registry — always required
SCHEMA_REGISTRY_URL=https://psrc-xxxxx.us-east-2.aws.confluent.cloud
SR_API_KEY=your_sr_api_key
SR_API_SECRET=your_sr_api_secret

# AWS KMS — required for --demo csfle (or --demo all)
AWS_KMS_KEY_ARN=arn:aws:kms:us-east-1:123456789012:key/your-key-id
```

`python-dotenv` loads this automatically at module startup via `load_dotenv()`; no `--env-file` flag is needed.

1.7 Confluent Cloud prerequisites

Resource	Minimum
Schema Registry	Stream Governance **Advanced** (required for DEK Registry / CSFLE)
SR API key	`DeveloperRead` + `DeveloperWrite` on Schema Registry
Kafka cluster	Any type — Basic, Standard, Dedicated, or Enterprise
Kafka API key	`DeveloperRead` + `DeveloperWrite` on the cluster
AWS KMS key	Symmetric encrypt/decrypt key — required for `--demo csfle`
AWS credentials	`boto3`-compatible auth (env vars, `~/.aws/credentials`, IAM role, or AWS SSO — see `run-all-demos.sh`)

1.8 Core classes

1.8.1 `SchemaRegistryClient` (`schema_registry_client.py`)

A `requests.Session`-based REST client covering the full SR API surface used by the demos. Authenticates with HTTP Basic (SR API key/secret). Maintains an in-process `_cache: dict[int, dict]` keyed by schema ID to avoid redundant `GET /schemas/ids/{id}` round-trips. Also includes the `_read_varint()` helper used by `decode_header()` to skip the message-index varint array.

Method	HTTP endpoint
`get_types()`	`GET /schemas/types`
`get_subjects()`	`GET /subjects`
`get_versions(subject)`	`GET /subjects/{s}/versions`
`get_version(subject, version)`	`GET /subjects/{s}/versions/{v}`

Method	HTTP endpoint
`get_schema_by_id(id)`	`GET /schemas/ids/{id}` *(cached)*
`get_versions_for_schema(id)`	`GET /schemas/ids/{id}/versions`
`referenced_by(subject, version)`	`GET /subjects/{s}/versions/{v}/referencedby`
`register(subject, schema, ...)`	`POST /subjects/{s}/versions`
`create_dek(kek, subj, algo?, encrypted_key_material?)`	`POST /dek-registry/v1/keks/{name}/deks`
`get_dek(kek, subject)`	`GET /dek-registry/v1/keks/{name}/deks/{subject}`
`delete_version(subject, version)`	`DELETE /subjects/{s}/versions/{v}`
`delete_subject(subject)`	`DELETE /subjects/{s}` *(soft)*
`delete_subject_permanent(subject)`	`DELETE /subjects/{s}?permanent=true`
`test_compatibility(subject, schema)`	`POST /compatibility/subjects/{s}/versions/{v}`
`get_compatibility(subject?)`	`GET /config[/{s}]`
`set_compatibility(level, subject?)`	`PUT /config[/{s}]`
`encode(schema_id, payload)`	*(local)* packs Confluent wire format
`decode_header(data)`	*(local)* validates magic byte, extracts schema ID

1.8.2 `ProtoMessage` / `ProtoField` (`dynamic_protobuf_helpers.py`)

Pure-Python dataclasses that build and binary-encode proto3 schemas without `protoc` or generated stubs, using the `google.protobuf` runtime directly.

Schema building — `to_schema_string()` emits valid proto3 text; `_to_fdp()` builds an equivalent `descriptor_pb2.FileDescriptorProto` programmatically, mapping every `ProtoField` to a `FieldDescriptorProto` (scalar types via `_PROTO_SCALAR_TYPES`, message-type references via `_full_type_name()`, proto3 `optional` via synthetic oneofs ordered last per the proto spec).

Descriptor pool — `message_class()` creates a fresh `descriptor_pool.DescriptorPool()` per `ProtoMessage` instance, calls `_ensure_in_pool()` to register all imported dependencies in dependency order, then calls `message_factory.GetMessageClass()` to obtain a real `google.protobuf.Message` subclass. Using a per-instance pool prevents name conflicts between schema-evolution versions of the same message.

SerDes — `serialize(data)` calls `json_format.ParseDict(data, cls())` then `.SerializeToString()`. `deserialize(raw)` calls `cls.FromString(raw)` then `json_format.MessageToDict(..., preserving_proto_field_name=True)`. Both produce and consume real Protobuf binary — no JSON stand-in.

1.8.3 `KafkaProtobufSerializer` (`kafka_protobuf_serdes.py`)

Mirrors the Java `KafkaProtobufSerializer`. Resolves the SR subject from the topic, message name, and `is_key` flag using the configured `subject_name_strategy`, then either auto-registers the schema or looks it up, and finally calls `sr.encode()` to wrap the payload in the Confluent wire format. Maintains a module-level `_schema_id_to_message` registry so the deserializer can resolve message classes by schema ID. When a `field_encryptor` is provided along with `metadata` and `rule_set`, tagged fields are encrypted via `FieldEncryptor.encrypt_fields()` before Protobuf serialization.

1.8.4 `KafkaProtobufDeserializer` (`kafka_protobuf_serdes.py`)

Mirrors the Java `KafkaProtobufDeserializer`. Strips the wire-format header via `sr.decode_header()`, warms the schema cache via `get_schema_by_id()`, then either delegates to `specific_type.deserialize()` or looks up the message class from the module-level `_schema_id_to_message` registry populated by the serializer (DynamicMessage equivalent). When a `field_encryptor` is provided, tagged fields are automatically decrypted after deserialization using metadata from either the SR response or an in-process `_schema_id_to_csfle` cache.

1.8.5 `FieldEncryptor` / `get_encrypted_fields()` (`field_encryption.py`)

Implements AES-256-GCM field-level encryption for Confluent CSFLE via AWS KMS.

`get_encrypted_fields(metadata, rule_set)` — Inspects schema metadata property tags (e.g. `SensitiveRecord.ssn.tags = "PII"`) and rule set domain rules of type `ENCRYPT` to return the list of field names targeted for encryption.

`FieldEncryptor` — Manages Data Encryption Keys (DEKs) per subject and encrypts/decrypts individual string field values. Uses the Confluent DEK Registry for DEK storage and AWS KMS for KEK management:

1. **First encrypt** — generates a 256-bit DEK locally, encrypts it via `kms.encrypt()`, then registers the encrypted key material with `create_dek()`
2. **Subsequent calls** — `get_dek()` returns the encrypted DEK from the registry, decrypted via `boto3 kms.decrypt()`
3. **DEKs are cached** in-process per subject to avoid repeated KMS calls

Encrypted values use the envelope format: `magic(0xC0) + version(0x01) + nonce(12 bytes) + ciphertext+tag(N bytes)`, base64-encoded before storage in the Protobuf string field.

Method	Purpose
`encrypt_value(value, subject)`	Encrypt a single string → base64 ciphertext
`decrypt_value(encrypted, subject)`	Decrypt a base64 ciphertext → original string
`encrypt_fields(data, fields, subject)`	Batch-encrypt named fields in a data dict
`decrypt_fields(data, fields, subject)`	Batch-decrypt named fields in a data dict
`_encrypt_dek_via_kms(plaintext_key)`	Encrypt a DEK via AWS KMS `kms.encrypt()`
`_decrypt_dek_via_kms(encrypted_key)`	Decrypt a DEK via AWS KMS `kms.decrypt()`

1.8.6 `kafka_helpers.py`

Contains all Kafka broker interaction logic, isolated from the demo and Schema Registry code. Only used when running with `--mode full`.

Function	Purpose
`_base_kafka_config(cfg)`	Builds the shared `bootstrap.servers` / `SASL_SSL` / `PLAIN` config dict
`ensure_topics(cfg, topics)`	`AdminClient` → `list_topics()` → `create_topics()` (rf=3, partitions=6, idempotent)
`kafka_produce(cfg, topic, key, value)`	`Producer` → `produce()` + `flush()`
`kafka_consume_one(cfg, topic, group_id)`	`Consumer` → `subscribe()` → `poll()` loop + `commit()`

1.8.7 `utilities.py`

Function	Purpose
`setup_logging(log_file?)`	Reads `[tool.logging]` from `pyproject.toml` via `tomllib` and applies it with `logging.config.dictConfig()`. Falls back to a basic dual-handler setup (file + console) if the config is absent. Returns the root logger.
`get_config()`	Reads the seven environment variables (`BOOTSTRAP_SERVERS`, `KAFKA_API_KEY`, …, `AWS_KMS_KEY_ARN`) and returns `(cfg_dict, missing_keys)`.
`parse_args()`	Defines the `--mode`, `--demo`, and `--run-id` CLI flags via `argparse` and returns the parsed `Namespace`.

1.8.8 `demos.py`

Contains all nine demo functions extracted from the former monolithic `main.py`. Each function receives a `SchemaRegistryClient`, an optional Kafka config dict (for `--mode full`), and/or the `run_id` suffix. The module has its own logger via `setup_logging()`.

Function	Demo
`demo_basic()`	1 — Basic Serializer & Deserializer
`demo_delete_protection()`	2 — Reference-Deletion Protection
`demo_evolution()`	3 — Schema Evolution
`demo_oneof()`	4 — Multiple Event Types / `oneOf`
`demo_null_handling()`	5 — Null-Value Handling

Function	Demo
`demo_compatibility()`	6 — Compatibility Rules
`demo_types()`	7 — Schema Types
`demo_strategies()`	8 — Subject Name Strategies
`demo_csfle()`	9 — Client-Side Field Level Encryption

1.9 Logging

Configured in `pyproject.toml` under `[tool.logging]` and loaded at startup by `utilities.setup_logging()`:

Handler	Level	Output
`console`	`DEBUG`	stdout
`file`	`INFO`	`cc-python-protobuf-csfle-example.log` (overwritten each run, mode `w`)

Log format: `YYYY-MM-DD HH:MM:SS - LEVEL - function_name - message`

The fallback log filename and format are defined as typed `Final` constants in `constants.py` (`DEFAULT_TOOL_LOG_FILE`, `DEFAULT_TOOL_LOG_FORMAT`).

1.10 Wire format

Every serialized Kafka message uses the Confluent wire format— the envelope that wraps protobuf binary before it goes onto a Kafka topic:

```
┌──────────┬──────────────────────┬──────────────────────┬──────────────
──────────┐
│  magic   │  schema_id           │  message index       │  payload
│
│  1 byte  │  4 bytes (big-endian)│  varint array        │  N bytes
│
│  0x00    │  e.g. 0x00000042 = 66│  0x00 (first msg)    │  Protobuf
binary      │
└──────────┴──────────────────────┴──────────────────────┴──────────────
──────────┘
```

Segment	Bytes	Purpose
Magic byte	`0x00`	Identifies this as a Confluent Schema Registry message
Schema ID	4 bytes, big-endian	Points to the schema stored in Schema Registry
Message index	Variable (varint-encoded)	Tells which message type in the .proto is encoded
Payload	Remaining bytes	The actual protobuf binary

1.10.1 Why a message index?

A single `.proto` file (or schema) can define multiple message types:

```
message Customer { ... }   // index [0]
message Product { ... }   // index [1]
message Order { ... }    // index [2]
```

The message index tells the consumer which one was serialized. When there's only one message (the common case), the index is an empty array encoded as the single byte `0x00`.

1.10.2 What the code does

In `schema_registry_client.py`:

`encode()` builds the envelope:

```
struct.pack(">bI", 0x00, schema_id) + b"\x00" + payload
             magic┘     └schema ID  └message index └payload
```

Then appends the protobuf binary after it.

`decode_header()` does the reverse:

- Validates the magic byte is `0x00`
- Unpacks the 4-byte schema ID
- Skips past the message-index varint array using `_read_varint()`
- Returns the remaining bytes as the protobuf payload

This envelope is what makes Schema Registry-aware consumers (in any language) able to look up the correct schema and deserialize the message automatically.

2.0 Running the demos

```
./run-demo.sh --profile=<SSO_PROFILE_NAME> \
              --mode=<schema-only|full> \
              [--demo=
<all|basic|delete|evolution|oneof|null|compat|types|strategies|csfle>] \
              [--run-id=<any string, e.g. "test1">]
```

Argument	Required	Choice	Default	Description

Argument	Required	Choice	Default	Description
`--profile`	✅	---	---	The AWS SSO profile name. Passed directly to `aws sso login` and `aws2-wrap` for authentication, and used to resolve `AWS_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`, which are then exported as `TF_VAR_aws_region`, `TF_VAR_aws_access_key_id`, `TF_VAR_aws_secret_access_key`, and `TF_VAR_aws_session_token` for Terraform, respectively.
`--mode`	✅	`schema-only`, `full`	`schema-only`	SR-only or full Kafka round-trip
`--demo`	❌	`all` `basic` `delete` `evolution` `oneof` `null` `compat` `types` `strategies` `csfle`	`all`	Which demo to run
`--run-id`	❌	any string	random 8-char UUID prefix	Appended to every topic and subject name to prevent collisions across runs

> All required flags must be provided; if a required flag is missing, the script exits with code `85`.

In `--mode=full`, the app calls `ensure_topics()` via `AdminClient` to pre-create all five required topics before any produce calls. Confluent Cloud mandates `replication_factor=3`; existing topics are silently skipped. Schema registration and Kafka produce/consume are fully integrated in both modes, but only in `full` mode do the messages actually go to Kafka. In `schema-only` mode, the app still registers schemas and prints the resulting wire-format bytes to the console, but does not interact with Kafka at all.

Prerequisites: AWS CLI v2, `aws2-wrap` (`pip install aws2-wrap`), and a configured AWS SSO profile with access to the KMS key.

2.1 Demos

2.1.1 Demo 1 — Basic Serializer & Deserializer (`--demo basic`)

Builds two `ProtoMessage` objects (`OtherRecord` and `MyRecord`), registers them in dependency order (referenced schema first), and serializes a message into the Confluent wire format. Prints the magic byte

(`0x00`), schema ID, and full hex payload. Deserializes back with `KafkaProtobufDeserializer` using `specific_type=my_record`. In full mode the wire bytes are produced to and consumed from Kafka.

Topics: `testproto-{run_id}`
Subjects: `other-{run_id}.proto`, `testproto-{run_id}-value`

2.1.2 Demo 2 — Reference-Deletion Protection (`--demo delete`)

Demonstrates that Schema Registry rejects deletion of a schema that is referenced by another. Calls `referenced_by()` to show the dependency graph, attempts to delete the leaf subject (expects a `RuntimeError`), then shows the correct order: delete the referencing subject first, then the referenced one.

2.1.3 Demo 3 — Schema Evolution (`--demo evolution`)

Registers a v1 `MyRecord` schema (`id`, `amount`), sets `BACKWARD_TRANSITIVE` compatibility on the subject via `PUT /config/{subject}`, then registers v2 with an added `customer_id` field. Calls `test_compatibility()` before registration to verify safety. In full mode, produces both schema versions to the same topic and consumes them.

Topics: `transactions-proto-{run_id}`
Subjects: `transactions-proto-{run_id}-value`

2.1.4 Demo 4 — Multiple Event Types / `oneOf` (`--demo oneof`)

Builds four schemas: `Customer`, `Product`, `Order`, and an `AllTypes` wrapper that holds all three under a `oneof oneof_type` field. Registers all four with cross-schema references, then serializes and routes three heterogeneous events (`customer`, `product`, `order`) through a single topic.

Topics: `all-events-{run_id}`
Subjects: `Customer-{run_id}.proto`, `Product-{run_id}.proto`, `Order-{run_id}.proto`, `all-events-{run_id}-value`

2.1.5 Demo 5 — Null-Value Handling (`--demo null`)

Shows the recommended proto3 approach for nullable fields using the `optional` keyword on `ProtoField`. Serializes a partial record (name only) and a full record, then deserializes both. Also prints the pre-proto3 `google.protobuf.StringValue` wrapper alternative used with Confluent Connect's `wrapper.for.nullable=true` flag.

Topics: `nullables-{run_id}`

2.1.6 Demo 6 — Compatibility Rules (`--demo compat`)

Fetches the SR global compatibility level via `GET /config` and prints a reference table of all seven modes. Highlights the key Protobuf-vs-Avro distinction: adding a new *message type* (not just a field) breaks FORWARD compatibility, making `BACKWARD_TRANSITIVE` the recommended default.

2.1.7 Demo 7 — Schema Types & Return Types (`--demo types`)

Calls `GET /schemas/types` to list which schema types your SR instance supports, then prints a reference table mapping Avro / Protobuf / JSON Schema to their specific and generic Python return types. Documents the Python equivalents of the Java `specific.protobuf.value.type` and `derive.type` config properties.

2.1.8 Demo 8 — Subject Name Strategies (`--demo strategies`)

Registers the same `Payment` schema under all three subject name strategies and prints the resulting subject name for each. Also documents both reference subject name strategies.

Strategy	Resulting subject
`TopicNameStrategy`	`payments-{run_id}-value`
`RecordNameStrategy`	`Payment`
`TopicRecordNameStrategy`	`payments-{run_id}-Payment`
`DefaultReferenceSubjectNameStrategy`	import path, e.g. `other.proto`
`QualifiedReferenceSubjectNameStrategy`	dotted form, e.g. `mypackage.myfile`

2.1.9 Demo 9 — Client-Side Field Level Encryption (`--demo csfle`)

Demonstrates Confluent CSFLE using AES-256-GCM field-level encryption with AWS KMS. Registers a KEK in the DEK Registry, defines a `SensitiveRecord` schema with PII fields (`ssn`, `email`), tags them via schema metadata, and encrypts them before Protobuf serialization using `FieldEncryptor`. Shows three views: original plaintext, deserialized without decryption (ciphertext visible), and deserialized with decryption (plaintext restored). Requires `AWS_KMS_KEY_ARN` and valid AWS credentials.

Topics: `csfle-{run_id}` **Subjects:** `csfle-{run_id}-value`

3.0 Cleanup

All topics and subjects are suffixed with `{run_id}`. To remove everything a specific run created:

```
# Soft-delete SR subjects
confluent schema-registry subject list \
  | grep '<run_id>' \
  | xargs -I{} confluent schema-registry subject delete --subject {} --
force

# Delete Kafka topics
for topic in testproto transactions-proto all-events nullables payments
csfle; do
  confluent kafka topic delete "${topic}-<run_id>"
done
```

4.0 Notes

- **No standalone SR Python library.** There is no `confluent-schema-registry` PyPI package. `SchemaRegistryClient` calls the REST API directly via `requests` — this is the correct approach for Python.
- **Confluent Cloud requires `replication_factor=3`.** `ensure_topics()` always passes this value; any other value will be rejected by the cluster.
- **`BACKWARD_TRANSITIVE` is the right default for Protobuf.** Unlike Avro, adding a new *message type* (not just a field) breaks FORWARD compatibility.
- **`uv.lock` should be committed.** It pins every transitive dependency for fully reproducible installs across machines and CI.
- **Real Protobuf binary encoding.** `ProtoMessage.serialize()` and `deserialize()` use `google.protobuf.message_factory` with dynamically constructed `FileDescriptorProto` objects — no `protoc`, no generated stubs, real binary on the wire. `json_format.ParseDict` / `MessageToDict` bridge between plain Python dicts and `google.protobuf.Message` instances.

5.0 Resources

- Confluent Protobuf SerDes documentation
- Protocol Buffers (or Protobuf for short) documentation
- Protect Sensitive Data Using Client-Side Field Level Encryption on Confluent Cloud